

Graph Theory Notes

Jesse Oldroyd

July 2, 2026

2026-07-02

Graph Theory Notes

Graph Theory Notes

Jesse Oldroyd
July 2, 2026

Introduction to Graph Theory

Trees

2026-07-02

└ Outline

Introduction to Graph Theory

Trees

2026-07-02

Graph Theory Notes
└ Introduction to Graph Theory

Introduction to Graph Theory

Introduction to Graph Theory

- Much like group theory is the mathematical language of symmetry, graph theory provides languages for expressing relationships

Applications of graph theory

- Modeling networks
- Chemistry
- Machine learning
- Routing problems
- CYOA

2026-07-02

Graph Theory Notes

└ Introduction to Graph Theory

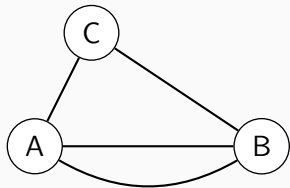
└ Networks and Relationships

- Much like group theory is the mathematical language of symmetry, graph theory provides languages for expressing relationships

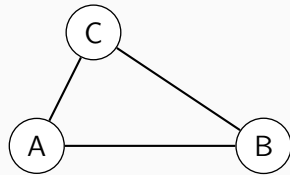
Applications of graph theory

- Modeling networks
- Chemistry
- Machine learning
- Routing problems
- CYOA

- A **graph** $\mathcal{G} = (V, E)$ is a set V of **vertices** together with a set E of **edges**. Two vertices are **adjacent** or neighbors if they are connected by an edge.
- A **simple graph** is a graph with at most one edge between any pair of vertices and no loops.



(a) A graph that is not simple.



(b) A graph that is simple.

- A **graph** $G = (V, E)$ is a set V of **vertices** together with a set E of **edges**. Two vertices are **adjacent** or neighbors if they are connected by an edge.
- A **simple graph** is a graph with at most one edge between any pair of vertices and no loops.



(a) A graph that is not simple.



(b) A graph that is simple.

- A friendship graph is a graph where each pair of vertices have exactly one neighbor (i.e., the "friend") in common.

Example (Friendship graphs)

Draw examples of friendship graphs.

2026-07-02

Graph Theory Notes

└ Introduction to Graph Theory

└ Friendship

Friendship

- A friendship graph is a graph where each pair of vertices have exactly one neighbor (i.e., the "friend") in common.

Example (Friendship graphs)

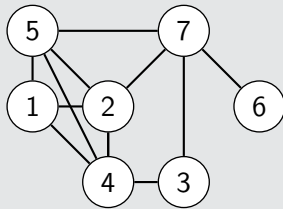
Draw examples of friendship graphs.

- The **degree** of a vertex is the number of edges adjacent to that vertex. A **path** between two vertices is a series of distinct, adjacent vertices that connects the vertices. A **cycle** is a path that starts and ends at the same vertex.

Example (Finding degrees, paths and cycles)

Find the following using the graph below:

1. The degree of each vertex.
2. The number of paths from vertex 1 to vertex 6.
3. The length of the shortest path from vertex 1 to vertex 6.



1. The degree of each vertex.
2. The number of paths from vertex 1 to vertex 6.
3. The length of the shortest path from vertex 1 to vertex 6.



Example (Scheduling tournaments)

Suppose six teams are playing in a tournament and each team must play exactly three others. How can this be done? What if only five teams are playing?

Example (Scheduling tournaments)

Suppose six teams are playing in a tournament and each team must play exactly three others. How can this be done? What if only five teams are playing?

An Impossibility Proof

Example (Not scheduling tournaments)

Prove that it's impossible to schedule a tournament for five teams such that each team plays exactly three others.

2026-07-02

Graph Theory Notes

└ Introduction to Graph Theory

└ An Impossibility Proof

Look at previous examples and try to find a relationship between degrees and edges.

Example (Utility graph)

Three houses A , B and C must be connected to three utilities E (electric), G (gas) and W (water). Each house must be connected to each utility exactly once. What graph represents this situation?

Example (Utility graph)

Three houses A , B and C must be connected to three utilities E (electric), G (gas) and W (water). Each house must be connected to each utility exactly once. What graph represents this situation?

- A **bipartite graph** is a graph with one group of m vertices and another group of n vertices such that each vertex is adjacent to all of the vertices in the other group and adjacent to none of the vertices in their own group. This graph is denoted by $K_{m,n}$.
- The utility graph is $K_{3,3}$.

- A **bipartite graph** is a graph with one group of m vertices and another group of n vertices such that each vertex is adjacent to all of the vertices in the other group and adjacent to none of the vertices in their own group. This graph is denoted by $K_{m,n}$.
- The utility graph is $K_{3,3}$.

Example (Employment)

Suppose that four employers (labeled A, B, C, D) are recruiting from a shared pool of four applicants (labeled $1, 2, 3, 4$). The employers and applicants have ranked each other as follows:

Applicant	Employer rankings	Employer	Applicant rankings
1	A, B, C, D	A	4, 2, 1, 3
2	A, B, D, C	B	2, 1, 3, 4
3	B, A, C, D	C	1, 3, 4, 2
4	D, A, B, C	D	1, 3, 2, 4

Employers will make offers to applicants, but applicants are allowed to change their minds if a better offer comes along. Is there a *stable matching*, i.e., a situation where no applicant will change their mind?

2026-07-02

Example (Employment)

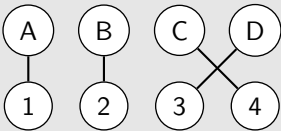
Suppose that four employers (labeled A, B, C, D) are recruiting from a shared pool of four applicants (labeled $1, 2, 3, 4$). The employers and applicants have ranked each other as follows:

Applicant	Employer rankings	Employer	Applicant rankings
1	A, B, C, D	A	4, 2, 1, 3
2	A, B, D, C	B	2, 1, 3, 4
3	B, A, C, D	C	1, 3, 4, 2
4	D, A, B, C	D	1, 3, 2, 4

Employers will make offers to applicants, but applicants are allowed to change their minds if a better offer comes along. Is there a *stable matching*, i.e., a situation where no applicant will change their mind?

An example of an unstable matching is the following:

Figure 2: A matching that is not stable.



This is because A prefers 2 to 1 and 2 prefers A to B . Therefore, both are inclined to switch.

- The previous problem can be solved by an algorithm. For each round:
 1. Each employer with an open position makes an offer to the best candidate that they haven't made an offer to yet.
 2. Each applicant evaluates their offers and tentatively selects the best offer while rejecting the others.
 3. If there are any open positions remaining, then repeat this process in a new round of offers.

- The previous problem can be solved by an algorithm. For each round:
 1. Each employer with an open position makes an offer to the best candidate that they haven't made an offer to yet.
 2. Each applicant evaluates their offers and tentatively selects the best offer while rejecting the others.
 3. If there are any open positions remaining, then repeat this process in a new round of offers.

This is known as the Gale-Shapley algorithm. It had previously been used to match residents with hospitals.

2026-07-02

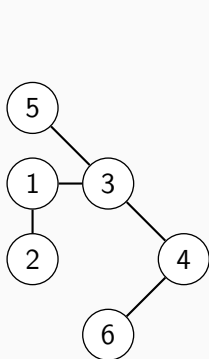
Graph Theory Notes
└ Trees

Trees

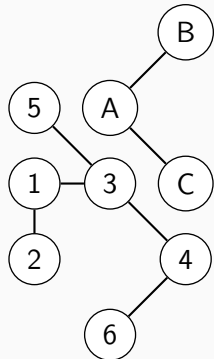
Trees

Definition and Examples

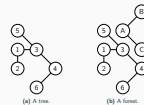
- A **tree** is a connected, acyclic graph (i.e., a graph that has no cycles).
- A **forest** is a collection of trees.



(a) A tree.



(b) A forest.



Trees are often useful as *data structures* in computer science. For instance, file directories on a computer can be modeled using trees.

- Trees have the special property that removing an edge (any edge!) disconnects the graph.
- There is also another nice property of the edges of a tree.

Example (Counting edges)

Draw several trees. How many edges do they have? What's the pattern?

2026-07-02

Graph Theory Notes

└ Trees

└ Edges and Trees

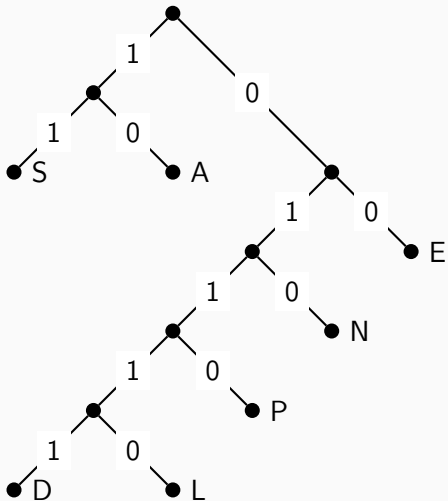
- Trees have the special property that removing an edge (any edge!) disconnects the graph.
- There is also another nice property of the edges of a tree.

Example (Counting edges)

Draw several trees. How many edges do they have? What's the pattern?

Huffman Code

- Trees are often useful as *data structures*. A **Huffman code** uses a tree to determine an optimal way to store characters in binary:



Example (Representing a maze as a tree)

Represent the pictured maze as a tree. How can you solve the maze using the tree?

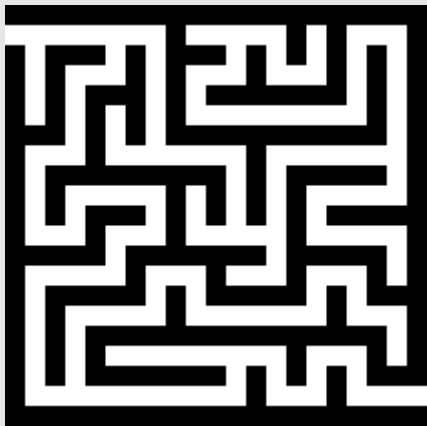


Figure 4: The maze.

2026-07-02

└ Trees

└ Mazes and Trees

Example (Representing a maze as a tree)

Represent the pictured maze as a tree. How can you solve the maze using the tree?

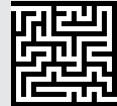


Figure 4: The maze.

- Solving the maze corresponds to finding the shortest path between S (the **root**) and F (the **target**) in the tree.
- Such paths can be found using **breadth-first search**:
 1. Add the root to the queue. While the queue is nonempty, do the following:
 2. Set the current vertex equal to the first vertex in the queue, removing it from the queue.
 3. If the current vertex is the target, then set the current vertex as the parent of the target and erase the queue.
 4. If the current vertex is not the target, then do the following for each neighbor:
 - 4.1 set the parent of the neighbor to the current vertex.
 - 4.2 add the neighbor to the queue.

- Solving the maze corresponds to finding the shortest path between S (the **root**) and F (the **target**) in the tree.
- Such paths can be found using **breadth-first search**:
 1. Add the root to the queue. While the queue is nonempty, do the following:
 2. Set the current vertex equal to the first vertex in the queue, removing it from the queue.
 3. If the current vertex is the target, then set the current vertex as the parent of the target and erase the queue.
 4. If the current vertex is not the target, then do the following for each neighbor:
 - 4.1 set the parent of the neighbor to the current vertex.
 - 4.2 add the neighbor to the queue.

Solving a maze graph

Apply the previous algorithm to find the shortest path from S to F in the following maze graph:

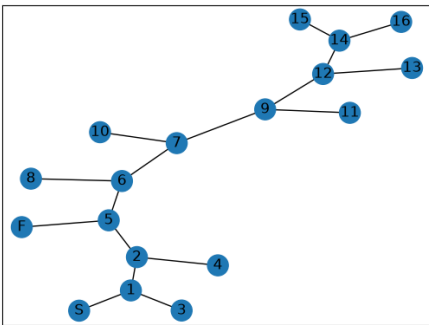


Figure 5: Maze graph.

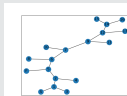


Figure 5: Maze graph.

BFS algorithm:

1. Add the root to the queue. While the queue is nonempty, do the following:
2. Set the current vertex equal to the first vertex in the queue, removing it from the queue.
3. If the current vertex is the target, then set the current vertex as the parent of the target and erase the queue.
4. If the current vertex is not the target, then do the following for each neighbor:
 - 4.1 set the parent of the neighbor to the current vertex.
 - 4.2 add the neighbor to the queue.